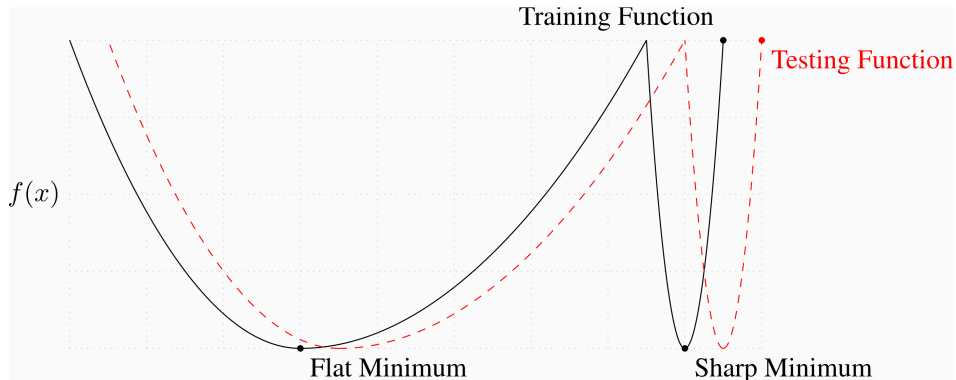


Оптимизация нейронных сетей

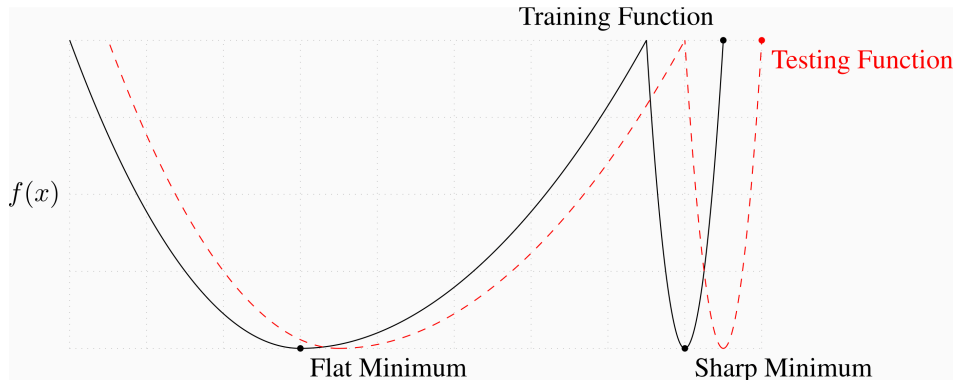
Семинар

ФКН ВШЭ

Плоские минимумы vs острые минимумы



Плоские минимумы vs острые минимумы



i Question

Что плохого в остром минимуме?

Оптимизация с учётом остроты (SAM) ¹

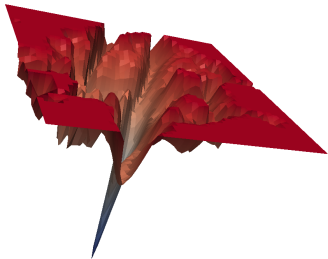


Figure 1: Острый минимум, в который сошёлся ResNet при обучении с SGD.

¹Foret, Pierre, et al. "Sharpness-aware minimization for efficiently improving generalization." (2020)

Оптимизация с учётом остроты (SAM) ¹

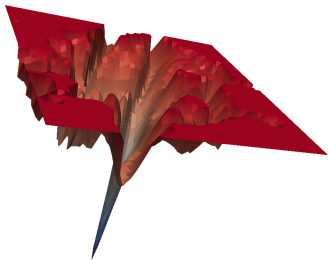


Figure 1: Острый минимум, в который сошёлся ResNet при обучении с SGD.

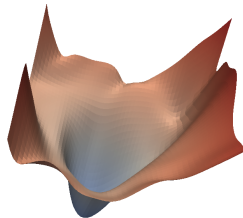


Figure 2: Широкий (плоский) минимум, в который сошёлся тот же ResNet при обучении с SAM.

¹Foret, Pierre, et al. "Sharpness-aware minimization for efficiently improving generalization." (2020)

Оптимизация с учётом остроты (SAM) ¹

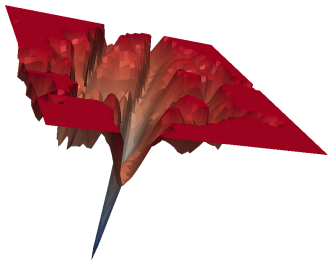


Figure 1: Острый минимум, в который сошёлся ResNet при обучении с SGD.

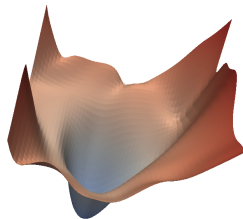


Figure 2: Широкий (плоский) минимум, в который сошёлся тот же ResNet при обучении с SAM.

Оптимизация с учётом остроты (SAM) — это процедура, направленная на улучшение обобщающей способности модели за счёт одновременной минимизации значения функции потерь и **остроты ландшафта функции потерь**.

¹Foret, Pierre, et al. "Sharpness-aware minimization for efficiently improving generalization." (2020)

Постановка задачи обучения

Обучающая выборка, полученная i.i.d. из распределения D :

$$S = \{(x_i, y_i)\}_{i=1}^n,$$

где x_i — вектор признаков, y_i — метка.

Постановка задачи обучения

Обучающая выборка, полученная i.i.d. из распределения D :

$$S = \{(x_i, y_i)\}_{i=1}^n,$$

где x_i — вектор признаков, y_i — метка.

Функция потерь на обучающей выборке:

$$L_S = \frac{1}{n} \sum_{i=1}^n l(w, x_i, y_i),$$

где l — функция потерь для одного объекта, w — параметры модели.

Постановка задачи обучения

Обучающая выборка, полученная i.i.d. из распределения D :

$$S = \{(x_i, y_i)\}_{i=1}^n,$$

где x_i — вектор признаков, y_i — метка.

Функция потерь на обучающей выборке:

$$L_S = \frac{1}{n} \sum_{i=1}^n l(w, x_i, y_i),$$

где l — функция потерь для одного объекта, w — параметры модели.

Функция потерь на генеральной совокупности:

$$L_D = \mathbb{E}_{(x,y)}[l(w, x, y)]$$

Что такое острота?

i Theorem

Для любого $\rho > 0$ с высокой вероятностью по обучающей выборке S , порождённой распределением D ,

$$L_D(w) \leq \max_{\|\epsilon\|_2 \leq \rho} L_S(w + \epsilon) + h(\|w\|_2^2 / \rho^2),$$

где $h : \mathbb{R}_+ \rightarrow \mathbb{R}_+$ — строго возрастающая функция (при некоторых технических условиях на $L_D(w)$).

Что такое острота?

i Theorem

Для любого $\rho > 0$ с высокой вероятностью по обучающей выборке S , порождённой распределением D ,

$$L_D(w) \leq \max_{\|\epsilon\|_2 \leq \rho} L_S(w + \epsilon) + h(\|w\|_2^2 / \rho^2),$$

где $h : \mathbb{R}_+ \rightarrow \mathbb{R}_+$ — строго возрастающая функция (при некоторых технических условиях на $L_D(w)$).

Прибавляя и вычитая $L_S(w)$:

$$\left[\max_{\|\epsilon\|_2 \leq \rho} L_S(w + \epsilon) - L_S(w) \right] + L_S(w) + h(\|w\|_2^2 / \rho^2)$$

Слагаемое в квадратных скобках отражает **остроту** L_S в точке w : оно показывает, насколько быстро можно увеличить функцию потерь на обучающей выборке, сдвинув параметры из w в соседнюю точку.

Оптимизация с учётом остроты (SAM)

Функцию h заменяют более простой константой λ . Авторы предлагают выбирать значения параметров, решая следующую задачу оптимизации с учётом остроты (SAM):

$$\min_w L_S^{SAM}(w) + \lambda \|w\|_2^2 \quad \text{где} \quad L_S^{SAM}(w) \triangleq \max_{\|\epsilon\|_p \leq \rho} L_S(w + \epsilon),$$

с гиперпараметром $\rho \geq 0$ и $p \in [1, \infty]$ (небольшое обобщение, хотя $p = 2$ эмпирически является лучшим выбором).

Как минимизировать L_S^{SAM} ?

Для минимизации L_S^{SAM} используется эффективное приближение её градиента. Первый шаг — разложение $L_S(w + \epsilon)$ в ряд Тейлора первого порядка:

$$\epsilon^*(w) \triangleq \arg \max_{\|\epsilon\|_p \leq \rho} L_S(w + \epsilon) \approx \arg \max_{\|\epsilon\|_p \leq \rho} L_S(w) + \epsilon^T \nabla_w L_S(w) = \arg \max_{\|\epsilon\|_p \leq \rho} \epsilon^T \nabla_w L_S(w).$$

Как минимизировать L_S^{SAM} ?

Для минимизации L_S^{SAM} используется эффективное приближение её градиента. Первый шаг — разложение $L_S(w + \epsilon)$ в ряд Тейлора первого порядка:

$$\epsilon^*(w) \triangleq \arg \max_{\|\epsilon\|_p \leq \rho} L_S(w + \epsilon) \approx \arg \max_{\|\epsilon\|_p \leq \rho} L_S(w) + \epsilon^T \nabla_w L_S(w) = \arg \max_{\|\epsilon\|_p \leq \rho} \epsilon^T \nabla_w L_S(w).$$

Последнее выражение — это просто $\arg \max$ скалярного произведения векторов ϵ и $\nabla_w L_S(w)$, и хорошо известно, какой аргумент его максимизирует:

$$\hat{\epsilon}(w) = \rho \operatorname{sign}(\nabla_w L_S(w)) |\nabla_w L_S(w)|^{q-1} / \left(\|\nabla_w L_S(w)\|_q^q \right)^{1/p},$$

где $1/p + 1/q = 1$.

Как минимизировать L_S^{SAM} ?

Для минимизации L_S^{SAM} используется эффективное приближение её градиента. Первый шаг — разложение $L_S(w + \epsilon)$ в ряд Тейлора первого порядка:

$$\epsilon^*(w) \triangleq \arg \max_{\|\epsilon\|_p \leq \rho} L_S(w + \epsilon) \approx \arg \max_{\|\epsilon\|_p \leq \rho} L_S(w) + \epsilon^T \nabla_w L_S(w) = \arg \max_{\|\epsilon\|_p \leq \rho} \epsilon^T \nabla_w L_S(w).$$

Последнее выражение — это просто $\arg \max$ скалярного произведения векторов ϵ и $\nabla_w L_S(w)$, и хорошо известно, какой аргумент его максимизирует:

$$\hat{\epsilon}(w) = \rho \operatorname{sign}(\nabla_w L_S(w)) |\nabla_w L_S(w)|^{q-1} / \left(\|\nabla_w L_S(w)\|_q^q \right)^{1/p},$$

где $1/p + 1/q = 1$.

Таким образом,

$$\begin{aligned} \nabla_w L_S^{SAM}(w) &\approx \nabla_w L_S(w + \hat{\epsilon}(w)) = \left. \frac{d(w + \hat{\epsilon}(w))}{dw} \nabla_w L_S(w) \right|_{w + \hat{\epsilon}(w)} \\ &= \nabla_w L_S(w)|_{w + \hat{\epsilon}(w)} + \left. \frac{d\hat{\epsilon}(w)}{dw} \nabla_w L_S(w) \right|_{w + \hat{\epsilon}(w)} \end{aligned}$$

Оптимизация с учётом остроты (SAM)

Современные фреймворки легко вычисляют описанное приближение. Однако для ускорения вычислений члены второго порядка можно отбросить, получив:

$$\nabla_w L_S^{SAM}(w) \approx \nabla_w L_S(w) \Big|_{w+\hat{\epsilon}(w)}$$

Оптимизация с учётом остроты (SAM)

Современные фреймворки легко вычисляют описанное приближение. Однако для ускорения вычислений члены второго порядка можно отбросить, получив:

$$\nabla_w L_S^{SAM}(w) \approx \nabla_w L_S(w)|_{w+\hat{\epsilon}(w)}$$

Input: Training set $\mathcal{S} \triangleq \cup_{i=1}^n \{(\mathbf{x}_i, \mathbf{y}_i)\}$, Loss function $l : \mathcal{W} \times \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}_+$, Batch size b , Step size $\eta > 0$, Neighborhood size $\rho > 0$.

Output: Model trained with SAM

Initialize weights $\mathbf{w}_0, t = 0$;

while *not converged* **do**

 Sample batch $\mathcal{B} = \{(\mathbf{x}_1, \mathbf{y}_1), \dots (\mathbf{x}_b, \mathbf{y}_b)\}$;

 Compute gradient $\nabla_w L_{\mathcal{B}}(\mathbf{w})$ of the batch's training loss;

 Compute $\hat{\epsilon}(\mathbf{w})$ per equation 2;

 Compute gradient approximation for the SAM objective

 (equation 3): $\mathbf{g} = \nabla_w L_{\mathcal{B}}(\mathbf{w})|_{w+\hat{\epsilon}(w)}$;

 Update weights: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}$;

$t = t + 1$;

end

return $\mathbf{w}_t$

Algorithm 1: SAM algorithm

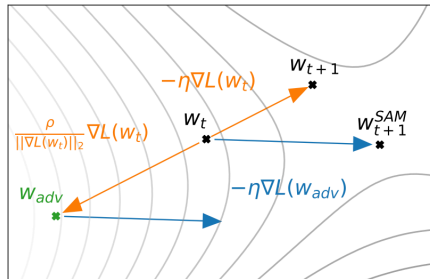


Figure 2: Schematic of the SAM parameter update.

Результаты SAM

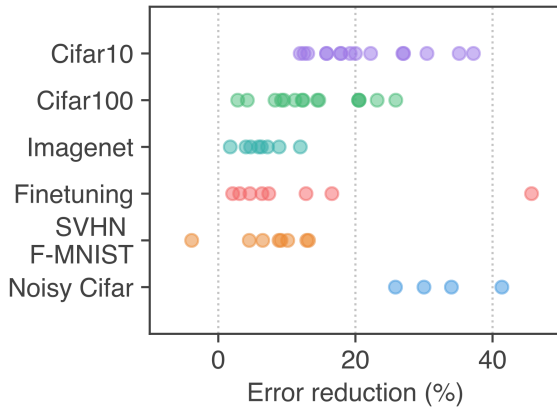


Figure 4: Снижение доли ошибок при переходе на SAM. Каждая точка — отдельный набор данных / модель / аугментация данных.

Почему методы редукции дисперсии не работают при обучении нейронных сетей? ²

- **SVRG / SAG** убедительно выигрывают на выпуклых задачах, однако на CIFAR-10 (LeNet-5) и ImageNet (ResNet-18) они не превосходят стандартный SGD.

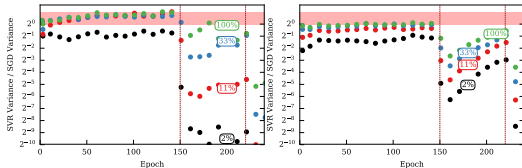


Figure 5: DenseNet

Figure 6: Small ResNet

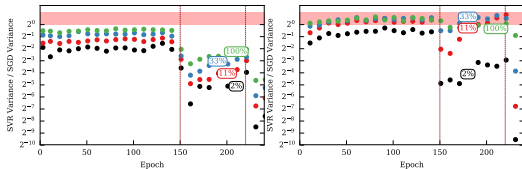


Figure 7: LeNet-5

Figure 8: ResNet-110

Почему методы редукции дисперсии не работают при обучении нейронных сетей? ²

- **SVRG / SAG** убедительно выигрывают на выпуклых задачах, однако на CIFAR-10 (LeNet-5) и ImageNet (ResNet-18) они не превосходят стандартный SGD.

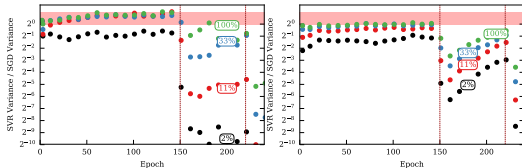


Figure 5: DenseNet

Figure 6: Small ResNet

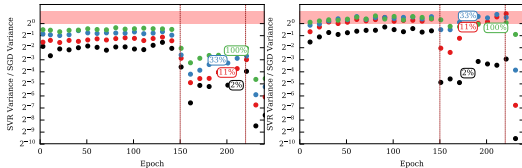


Figure 7: LeNet-5

Figure 8: ResNet-110

Почему методы редукции дисперсии не работают при обучении нейронных сетей? ²

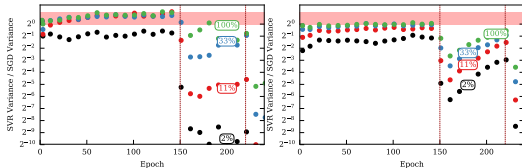


Figure 5: DenseNet

Figure 6: Small ResNet

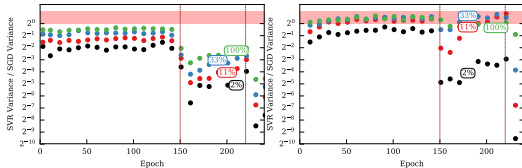


Figure 7: LeNet-5

Figure 8: ResNet-110

- **SVRG / SAG** убедительно выигрывают на выпуклых задачах, однако на CIFAR-10 (LeNet-5) и ImageNet (ResNet-18) они не превосходят стандартный SGD.
- Измеренное отношение «дисперсия SGD / дисперсия SVRG» остаётся $\lesssim 2$ для большинства слоёв — то есть реальное снижение шума минимально.

Почему методы редукции дисперсии не работают при обучении нейронных сетей? ²

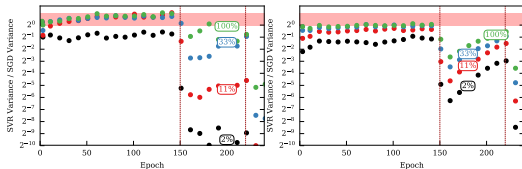


Figure 5: DenseNet

Figure 6: Small ResNet

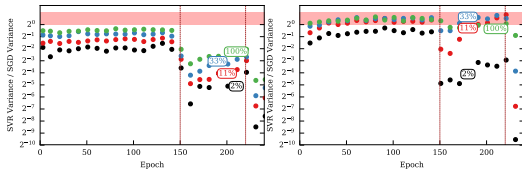


Figure 7: LeNet-5

Figure 8: ResNet-110

- **SVRG / SAG** убедительно выигрывают на выпуклых задачах, однако на CIFAR-10 (LeNet-5) и ImageNet (ResNet-18) они не превосходят стандартный SGD.
- Измеренное отношение «дисперсия SGD / дисперсия SVRG» остаётся $\lesssim 2$ для большинства слоёв — то есть реальное снижение шума минимально.

Почему методы редукции дисперсии не работают при обучении нейронных сетей? ²

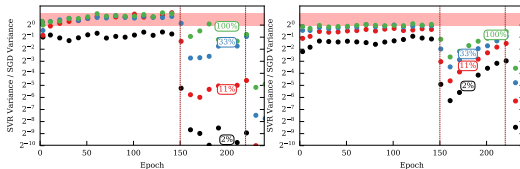


Figure 5: DenseNet

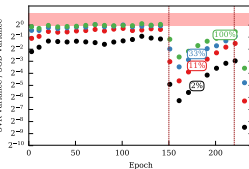


Figure 6: Small ResNet

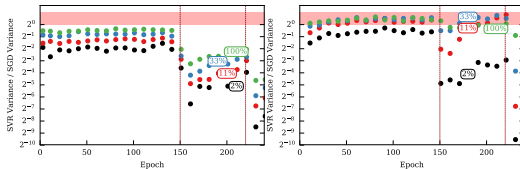


Figure 7: LeNet-5

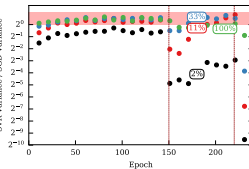


Figure 8: ResNet-110

- **SVRG / SAG** убедительно выигрывают на выпуклых задачах, однако на CIFAR-10 (LeNet-5) и ImageNet (ResNet-18) они не превосходят стандартный SGD.
- Измеренное отношение «дисперсия SGD / дисперсия SVRG» остаётся $\lesssim 2$ для большинства слоёв — то есть реальное снижение шума минимально.
- Возможные причины:

Почему методы редукции дисперсии не работают при обучении нейронных сетей? ²

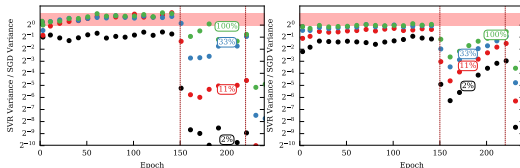


Figure 5: DenseNet

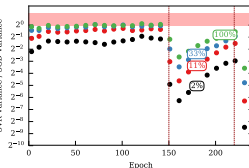


Figure 6: Small ResNet

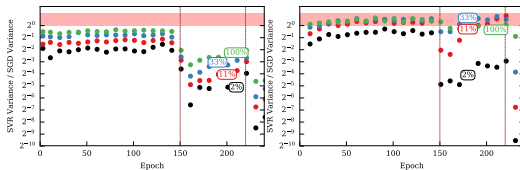


Figure 7: LeNet-5

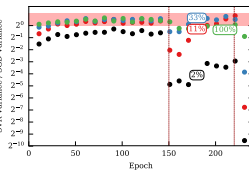


Figure 8: ResNet-110

- **SVRG / SAG** убедительно выигрывают на выпуклых задачах, однако на CIFAR-10 (LeNet-5) и ImageNet (ResNet-18) они не превосходят стандартный SGD.
- Измеренное отношение «дисперсия SGD / дисперсия SVRG» остаётся $\lesssim 2$ для большинства слоёв — то есть реальное снижение шума минимально.
- Возможные причины:
 - **Аугментация данных** делает опорный градиент g_{ref} устаревшим уже через несколько мини-батчей.

Почему методы редукции дисперсии не работают при обучении нейронных сетей? ²

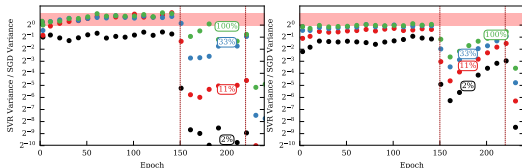


Figure 5: DenseNet

Figure 6: Small ResNet

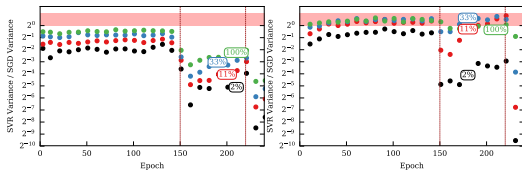


Figure 7: LeNet-5

Figure 8: ResNet-110

- **SVRG / SAG** убедительно выигрывают на выпуклых задачах, однако на CIFAR-10 (LeNet-5) и ImageNet (ResNet-18) они не превосходят стандартный SGD.
- Измеренное отношение «дисперсия SGD / дисперсия SVRG» остаётся $\lesssim 2$ для большинства слоёв — то есть реальное снижение шума минимально.
- Возможные причины:
 - **Аугментация данных** делает опорный градиент g_{ref} устаревшим уже через несколько мини-батчей.

Почему методы редукции дисперсии не работают при обучении нейронных сетей? ²

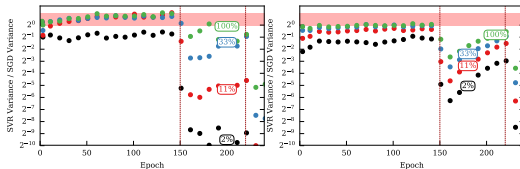


Figure 5: DenseNet

Figure 6: Small ResNet

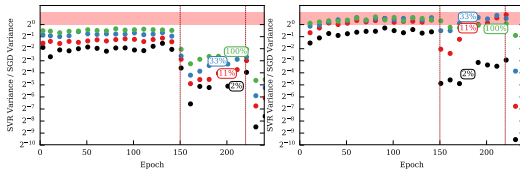


Figure 7: LeNet-5

Figure 8: ResNet-110

- **SVRG / SAG** убедительно выигрывают на выпуклых задачах, однако на CIFAR-10 (LeNet-5) и ImageNet (ResNet-18) они не превосходят стандартный SGD.
- Измеренное отношение «дисперсия SGD / дисперсия SVRG» остаётся $\lesssim 2$ для большинства слоёв — то есть реальное снижение шума минимально.
- Возможные причины:
 - **Аугментация данных** делает опорный градиент g_{ref} устаревшим уже через несколько мини-батчей.
 - **BatchNorm** и **Dropout** вносят внутреннюю стохастичность, которую прошлый g_{ref} не может компенсировать.

Почему методы редукции дисперсии не работают при обучении нейронных сетей? ²

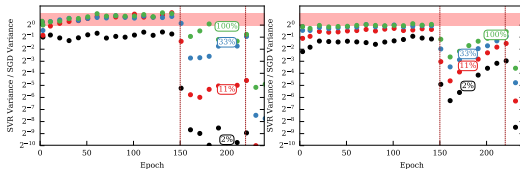


Figure 5: DenseNet

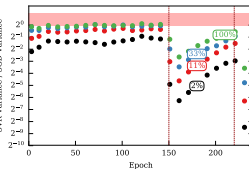


Figure 6: Small ResNet

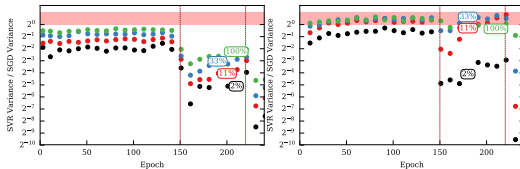


Figure 7: LeNet-5

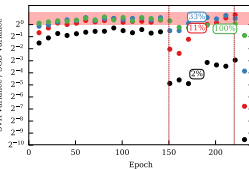


Figure 8: ResNet-110

- **SVRG / SAG** убедительно выигрывают на выпуклых задачах, однако на CIFAR-10 (LeNet-5) и ImageNet (ResNet-18) они не превосходят стандартный SGD.
- Измеренное отношение «дисперсия SGD / дисперсия SVRG» остаётся $\lesssim 2$ для большинства слоёв — то есть реальное снижение шума минимально.
- Возможные причины:
 - **Аугментация данных** делает опорный градиент g_{ref} устаревшим уже через несколько мини-батчей.
 - **BatchNorm** и **Dropout** вносят внутреннюю стохастичность, которую прошлый g_{ref} не может компенсировать.

Почему методы редукции дисперсии не работают при обучении нейронных сетей? ²

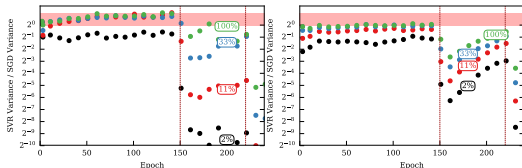


Figure 5: DenseNet

Figure 6: Small ResNet

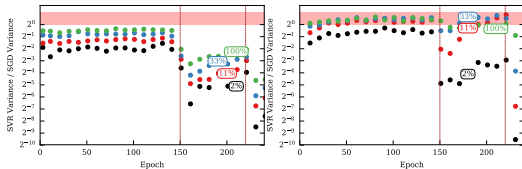


Figure 7: LeNet-5

Figure 8: ResNet-110

- **SVRG / SAG** убедительно выигрывают на выпуклых задачах, однако на CIFAR-10 (LeNet-5) и ImageNet (ResNet-18) они не превосходят стандартный SGD.
- Измеренное отношение «дисперсия SGD / дисперсия SVRG» остаётся $\lesssim 2$ для большинства слоёв — то есть реальное снижение шума минимально.
- Возможные причины:
 - **Аугментация данных** делает опорный градиент g_{ref} устаревшим уже через несколько мини-батчей.
 - **BatchNorm** и **Dropout** вносят внутреннюю стохастичность, которую прошлый g_{ref} не может компенсировать.
 - Дополнительный проход по всей выборке (для вычисления g_{ref}) поглощает потенциальную экономию на итерациях.

Почему методы редукции дисперсии не работают при обучении нейронных сетей? ²

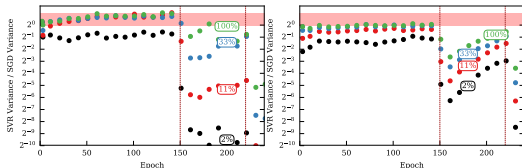


Figure 5: DenseNet

Figure 6: Small ResNet

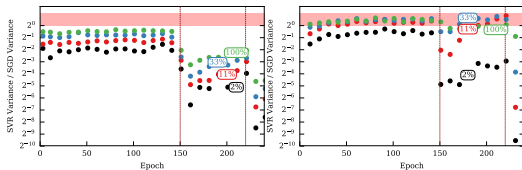


Figure 7: LeNet-5

Figure 8: ResNet-110

- **SVRG / SAG** убедительно выигрывают на выпуклых задачах, однако на CIFAR-10 (LeNet-5) и ImageNet (ResNet-18) они не превосходят стандартный SGD.
- Измеренное отношение «дисперсия SGD / дисперсия SVRG» остаётся $\lesssim 2$ для большинства слоёв — то есть реальное снижение шума минимально.
- Возможные причины:
 - **Аугментация данных** делает опорный градиент g_{ref} устаревшим уже через несколько мини-батчей.
 - **BatchNorm** и **Dropout** вносят внутреннюю стохастичность, которую прошлый g_{ref} не может компенсировать.
 - Дополнительный проход по всей выборке (для вычисления g_{ref}) поглощает потенциальную экономию на итерациях.

Почему методы редукции дисперсии не работают при обучении нейронных сетей? ²

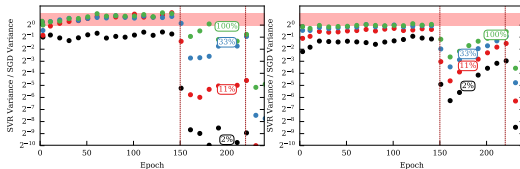


Figure 5: DenseNet

Figure 6: Small ResNet

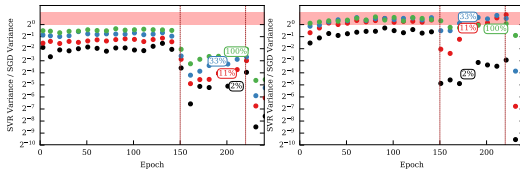


Figure 7: LeNet-5

Figure 8: ResNet-110

- **SVRG / SAG** убедительно выигрывают на выпуклых задачах, однако на CIFAR-10 (LeNet-5) и ImageNet (ResNet-18) они не превосходят стандартный SGD.
- Измеренное отношение «дисперсия SGD / дисперсия SVRG» остаётся $\lesssim 2$ для большинства слоёв — то есть реальное снижение шума минимально.
- Возможные причины:
 - **Аугментация данных** делает опорный градиент g_{ref} устаревшим уже через несколько мини-батчей.
 - **BatchNorm** и **Dropout** вносят внутреннюю стохастичность, которую прошлый g_{ref} не может компенсировать.
 - Дополнительный проход по всей выборке (для вычисления g_{ref}) поглощает потенциальную экономию на итерациях.
- «Потоковые» модификации SVRG, разработанные для работы с аугментацией, снижают теоретическое смещение, но всё равно уступают SGD и по времени, и по качеству.

Почему методы редукции дисперсии не работают при обучении нейронных сетей? ²

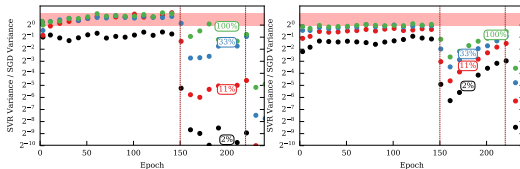


Figure 5: DenseNet

Figure 6: Small ResNet

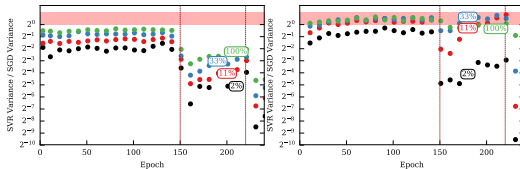


Figure 7: LeNet-5

Figure 8: ResNet-110

- **SVRG / SAG** убедительно выигрывают на выпуклых задачах, однако на CIFAR-10 (LeNet-5) и ImageNet (ResNet-18) они не превосходят стандартный SGD.
- Измеренное отношение «дисперсия SGD / дисперсия SVRG» остаётся $\lesssim 2$ для большинства слоёв — то есть реальное снижение шума минимально.
- Возможные причины:
 - **Аугментация данных** делает опорный градиент g_{ref} устаревшим уже через несколько мини-батчей.
 - **BatchNorm** и **Dropout** вносят внутреннюю стохастичность, которую прошлый g_{ref} не может компенсировать.
 - Дополнительный проход по всей выборке (для вычисления g_{ref}) поглощает потенциальную экономию на итерациях.
- «Потоковые» модификации SVRG, разработанные для работы с аугментацией, снижают теоретическое смещение, но всё равно уступают SGD и по времени, и по качеству.

Почему методы редукции дисперсии не работают при обучении нейронных сетей? ²

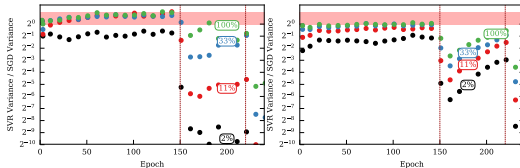


Figure 5: DenseNet

Figure 6: Small ResNet

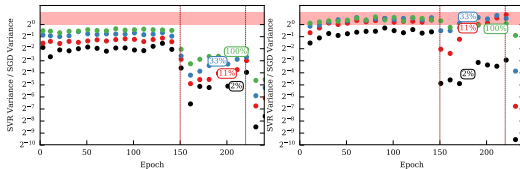


Figure 7: LeNet-5

Figure 8: ResNet-110

- **SVRG / SAG** убедительно выигрывают на выпуклых задачах, однако на CIFAR-10 (LeNet-5) и ImageNet (ResNet-18) они не превосходят стандартный SGD.
- Измеренное отношение «дисперсия SGD / дисперсия SVRG» остаётся $\lesssim 2$ для большинства слоёв — то есть реальное снижение шума минимально.
- Возможные причины:
 - **Аугментация данных** делает опорный градиент g_{ref} устаревшим уже через несколько мини-батчей.
 - **BatchNorm** и **Dropout** вносят внутреннюю стохастичность, которую прошлый g_{ref} не может компенсировать.
 - Дополнительный проход по всей выборке (для вычисления g_{ref}) поглощает потенциальную экономию на итерациях.
- «Потоковые» модификации SVRG, разработанные для работы с аугментацией, снижают теоретическое смещение, но всё равно уступают SGD и по времени, и по качеству.
- **Вывод:** Существующие методы редукции дисперсии непрактичны для современных глубоких сетей; будущие решения должны учитывать стохастическую природу архитектуры и данных (аугментация, BatchNorm, Dropout).

Связность минимумов³

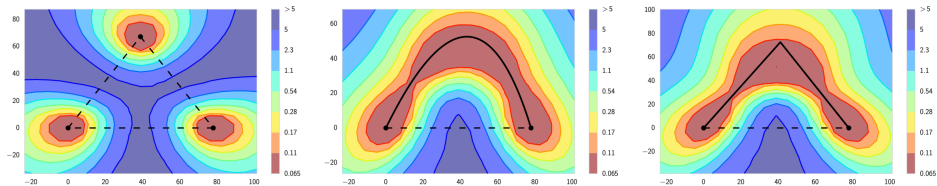


Figure 9: Поверхность l_2 -регуляризованных перекрёстных потерь на обучающей выборке для ResNet-164 на CIFAR-100 как функция весов сети в двумерном подпространстве. В каждой панели горизонтальная ось зафиксирована и привязана к оптимумам двух независимо обученных сетей. Вертикальная ось меняется между панелями по мере смены плоскостей (определения — в основном тексте). Слева: три оптимума для независимо обученных сетей. В центре и справа: квадратичная кривая Безье и ломаная с одним изломом, соединяющие два нижних оптимума левой панели вдоль пути почти постоянных потерь. Обратите внимание: в каждой панели прямой линейный путь между модами проходил бы через область высоких потерь.

³Garipov, T., Izmailov, P., Podoprikin, D., Vetrov, D. P., Wilson, A. G. (2018). Loss surfaces, mode connectivity, and fast ensembling of dnns. Advances in neural information processing systems, 31.

Процедура поиска кривой

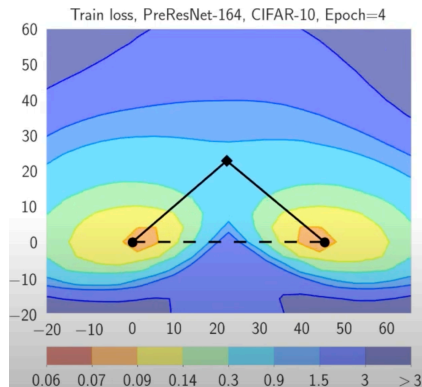
- Веса предобученных сетей:

$$\widehat{w}_1, \widehat{w}_2 \in \mathbb{R}^{|\text{net}|}$$

$$\phi_\theta(0) = \widehat{w}_1, \quad \phi_\theta(1) = \widehat{w}_2$$

$$\mathcal{L}(w)$$

$$\underset{\theta}{\text{minimize}} \quad \ell(\theta) = \int_0^1 \mathcal{L}(\phi_\theta(t)) dt = \mathbb{E}_{t \sim U(0,1)} \mathcal{L}(\phi_\theta(t))$$



Процедура поиска кривой

- Веса предобученных сетей:

$$\widehat{w}_1, \widehat{w}_2 \in \mathbb{R}^{|\text{net}|}$$

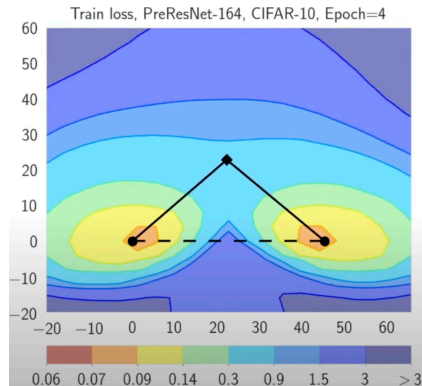
- Определяем параметрическую кривую:

$$\phi_\theta(\cdot) : [0, 1] \rightarrow \mathbb{R}^{|\text{net}|}$$

$$\phi_\theta(0) = \widehat{w}_1, \quad \phi_\theta(1) = \widehat{w}_2$$

$$\mathcal{L}(w)$$

$$\underset{\theta}{\text{minimize}} \quad \ell(\theta) = \int_0^1 \mathcal{L}(\phi_\theta(t)) dt = \mathbb{E}_{t \sim U(0,1)} \mathcal{L}(\phi_\theta(t))$$



Процедура поиска кривой

- Веса предобученных сетей:

$$\widehat{w}_1, \widehat{w}_2 \in \mathbb{R}^{|\text{net}|}$$

- Определяем параметрическую кривую:

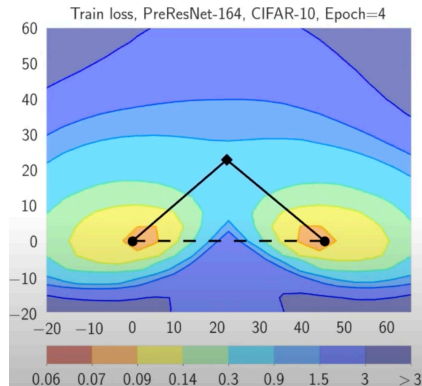
$$\phi_\theta(\cdot) : [0, 1] \rightarrow \mathbb{R}^{|\text{net}|}$$

$$\phi_\theta(0) = \widehat{w}_1, \quad \phi_\theta(1) = \widehat{w}_2$$

- Функция потерь нейронной сети:

$$\mathcal{L}(w)$$

$$\underset{\theta}{\text{minimize}} \quad \ell(\theta) = \int_0^1 \mathcal{L}(\phi_\theta(t)) dt = \mathbb{E}_{t \sim U(0,1)} \mathcal{L}(\phi_\theta(t))$$



Процедура поиска кривой

- Веса предобученных сетей:

$$\widehat{w}_1, \widehat{w}_2 \in \mathbb{R}^{|\text{net}|}$$

- Определяем параметрическую кривую:

$$\phi_\theta(\cdot) : [0, 1] \rightarrow \mathbb{R}^{|\text{net}|}$$

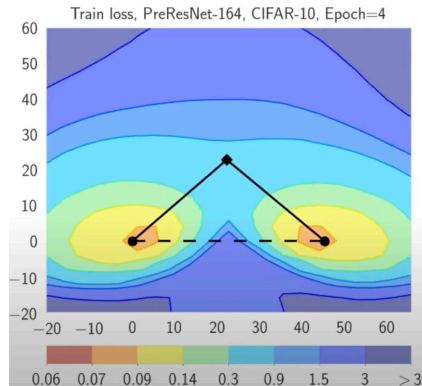
$$\phi_\theta(0) = \widehat{w}_1, \quad \phi_\theta(1) = \widehat{w}_2$$

- Функция потерь нейронной сети:

$$\mathcal{L}(w)$$

- Минимизируем усреднённые потери по θ :

$$\underset{\theta}{\text{minimize}} \quad \ell(\theta) = \int_0^1 \mathcal{L}(\phi_\theta(t)) dt = \mathbb{E}_{t \sim U(0,1)} \mathcal{L}(\phi_\theta(t))$$



Процедура поиска кривой

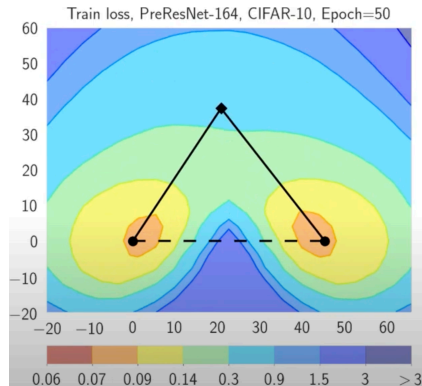
- Веса предобученных сетей:

$$\widehat{w}_1, \widehat{w}_2 \in \mathbb{R}^{|\text{net}|}$$

$$\phi_\theta(0) = \widehat{w}_1, \quad \phi_\theta(1) = \widehat{w}_2$$

$$\mathcal{L}(w)$$

$$\underset{\theta}{\text{minimize}} \quad \ell(\theta) = \int_0^1 \mathcal{L}(\phi_\theta(t)) dt = \mathbb{E}_{t \sim U(0,1)} \mathcal{L}(\phi_\theta(t))$$



Процедура поиска кривой

- Веса предобученных сетей:

$$\widehat{w}_1, \widehat{w}_2 \in \mathbb{R}^{|\text{net}|}$$

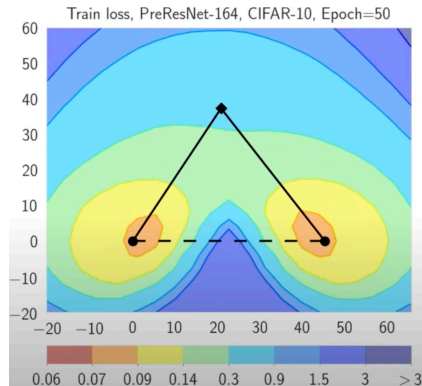
- Определяем параметрическую кривую:

$$\phi_\theta(\cdot) : [0, 1] \rightarrow \mathbb{R}^{|\text{net}|}$$

$$\phi_\theta(0) = \widehat{w}_1, \quad \phi_\theta(1) = \widehat{w}_2$$

$$\mathcal{L}(w)$$

$$\underset{\theta}{\text{minimize}} \quad \ell(\theta) = \int_0^1 \mathcal{L}(\phi_\theta(t)) dt = \mathbb{E}_{t \sim U(0,1)} \mathcal{L}(\phi_\theta(t))$$



Процедура поиска кривой

- Веса предобученных сетей:

$$\widehat{w}_1, \widehat{w}_2 \in \mathbb{R}^{|\text{net}|}$$

- Определяем параметрическую кривую:

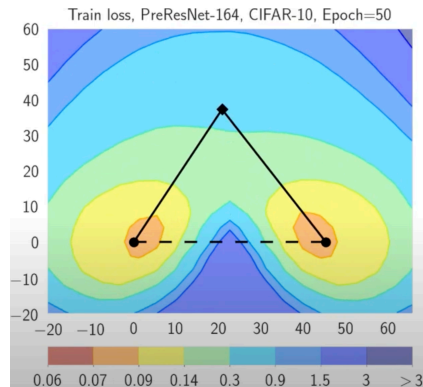
$$\phi_\theta(\cdot) : [0, 1] \rightarrow \mathbb{R}^{|\text{net}|}$$

$$\phi_\theta(0) = \widehat{w}_1, \quad \phi_\theta(1) = \widehat{w}_2$$

- Функция потерь нейронной сети:

$$\mathcal{L}(w)$$

$$\underset{\theta}{\text{minimize}} \quad \ell(\theta) = \int_0^1 \mathcal{L}(\phi_\theta(t)) dt = \mathbb{E}_{t \sim U(0,1)} \mathcal{L}(\phi_\theta(t))$$



Процедура поиска кривой

- Веса предобученных сетей:

$$\widehat{w}_1, \widehat{w}_2 \in \mathbb{R}^{|\text{net}|}$$

- Определяем параметрическую кривую:

$$\phi_\theta(\cdot) : [0, 1] \rightarrow \mathbb{R}^{|\text{net}|}$$

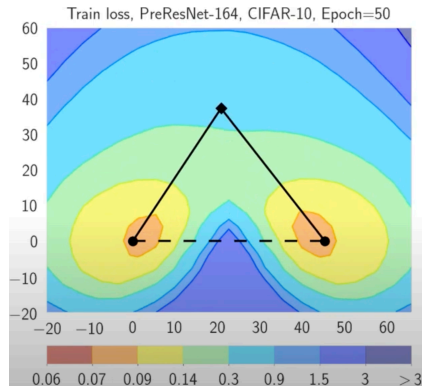
$$\phi_\theta(0) = \widehat{w}_1, \quad \phi_\theta(1) = \widehat{w}_2$$

- Функция потерь нейронной сети:

$$\mathcal{L}(w)$$

- Минимизируем усреднённые потери по θ :

$$\underset{\theta}{\text{minimize}} \quad \ell(\theta) = \int_0^1 \mathcal{L}(\phi_\theta(t)) dt = \mathbb{E}_{t \sim U(0,1)} \mathcal{L}(\phi_\theta(t))$$



Процедура поиска кривой

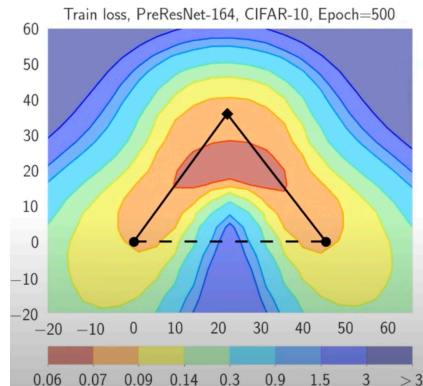
- Веса предобученных сетей:

$$\widehat{w}_1, \widehat{w}_2 \in \mathbb{R}^{|\text{net}|}$$

$$\phi_\theta(0) = \widehat{w}_1, \quad \phi_\theta(1) = \widehat{w}_2$$

$$\mathcal{L}(w)$$

$$\underset{\theta}{\text{minimize}} \quad \ell(\theta) = \int_0^1 \mathcal{L}(\phi_\theta(t)) dt = \mathbb{E}_{t \sim U(0,1)} \mathcal{L}(\phi_\theta(t))$$



Процедура поиска кривой

- Веса предобученных сетей:

$$\widehat{w}_1, \widehat{w}_2 \in \mathbb{R}^{|\text{net}|}$$

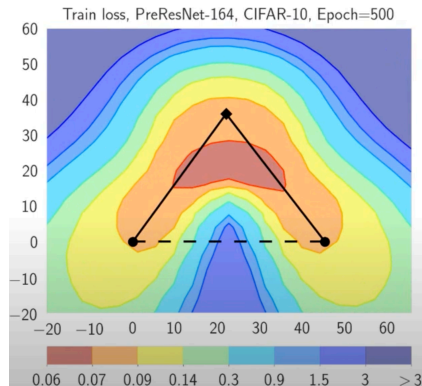
- Определяем параметрическую кривую:

$$\phi_\theta(\cdot) : [0, 1] \rightarrow \mathbb{R}^{|\text{net}|}$$

$$\phi_\theta(0) = \widehat{w}_1, \quad \phi_\theta(1) = \widehat{w}_2$$

$$\mathcal{L}(w)$$

$$\underset{\theta}{\text{minimize}} \quad \ell(\theta) = \int_0^1 \mathcal{L}(\phi_\theta(t)) dt = \mathbb{E}_{t \sim U(0,1)} \mathcal{L}(\phi_\theta(t))$$



Процедура поиска кривой

- Веса предобученных сетей:

$$\widehat{w}_1, \widehat{w}_2 \in \mathbb{R}^{|\text{net}|}$$

- Определяем параметрическую кривую:

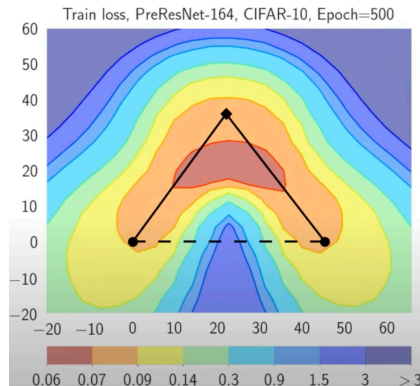
$$\phi_\theta(\cdot) : [0, 1] \rightarrow \mathbb{R}^{|\text{net}|}$$

$$\phi_\theta(0) = \widehat{w}_1, \quad \phi_\theta(1) = \widehat{w}_2$$

- Функция потерь нейронной сети:

$$\mathcal{L}(w)$$

$$\underset{\theta}{\text{minimize}} \quad \ell(\theta) = \int_0^1 \mathcal{L}(\phi_\theta(t)) dt = \mathbb{E}_{t \sim U(0,1)} \mathcal{L}(\phi_\theta(t))$$



Процедура поиска кривой

- Веса предобученных сетей:

$$\widehat{w}_1, \widehat{w}_2 \in \mathbb{R}^{|\text{net}|}$$

- Определяем параметрическую кривую:

$$\phi_\theta(\cdot) : [0, 1] \rightarrow \mathbb{R}^{|\text{net}|}$$

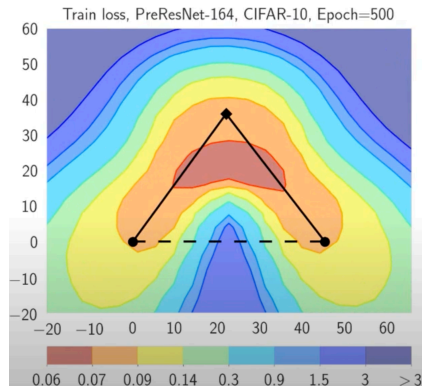
$$\phi_\theta(0) = \widehat{w}_1, \quad \phi_\theta(1) = \widehat{w}_2$$

- Функция потерь нейронной сети:

$$\mathcal{L}(w)$$

- Минимизируем усреднённые потери по θ :

$$\underset{\theta}{\text{minimize}} \quad \ell(\theta) = \int_0^1 \mathcal{L}(\phi_\theta(t)) dt = \mathbb{E}_{t \sim U(0,1)} \mathcal{L}(\phi_\theta(t))$$



- После достижения нулевых потерь на обучающей выборке веса продолжают эволюционировать в режиме случайного блуждания

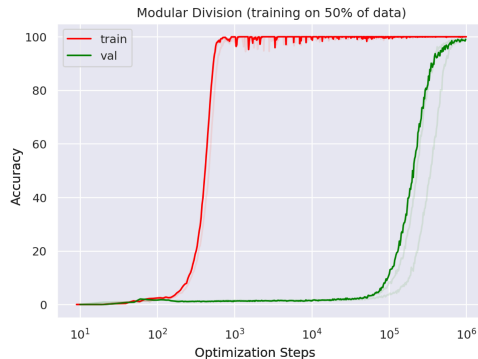


Figure 10: Грокинг: наглядный пример обобщения, возникающего значительно позже переобучения на алгоритмическом наборе данных.

- После достижения нулевых потерь на обучающей выборке веса продолжают эволюционировать в режиме случайного блуждания
- Возможно, они медленно смещаются к более широкому минимуму

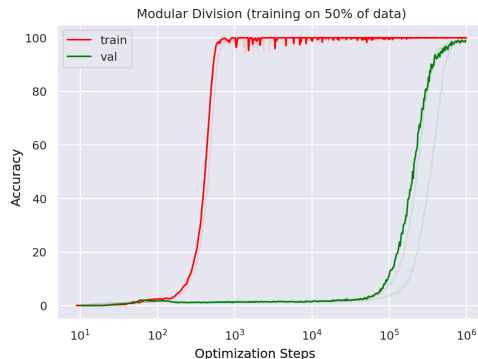


Figure 10: Грокинг: наглядный пример обобщения, возникающего значительно позже переобучения на алгоритмическом наборе данных.

- После достижения нулевых потерь на обучающей выборке веса продолжают эволюционировать в режиме случайного блуждания
- Возможно, они медленно смещаются к более широкому минимуму
- Недавно обнаруженный эффект грокинга подтверждает эту гипотезу

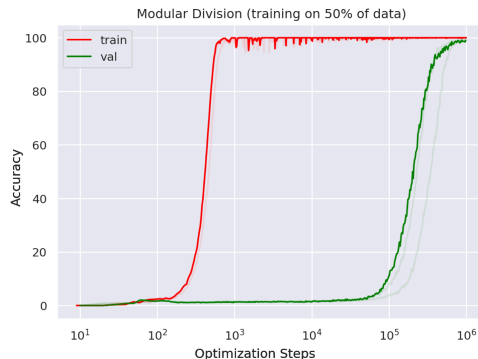


Figure 10: Грокинг: наглядный пример обобщения, возникающего значительно позже переобучения на алгоритмическом наборе данных.

⁴Power, Alethea, et al. "Grokking: Generalization beyond overfitting on small algorithmic datasets." (2022).

Двойной спуск⁵

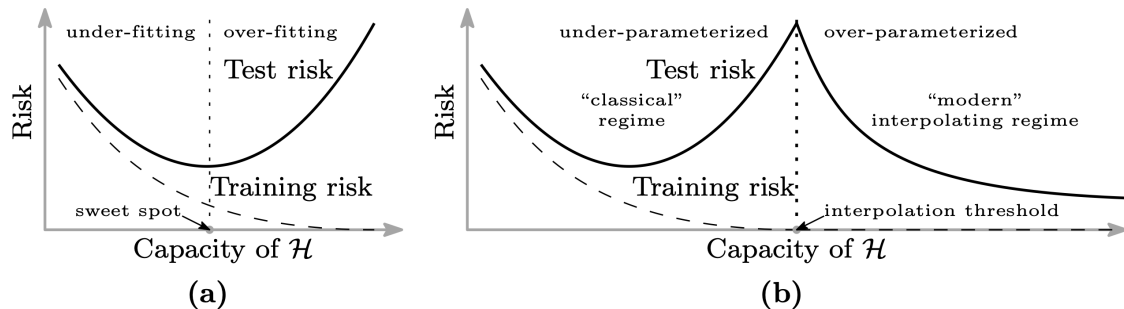
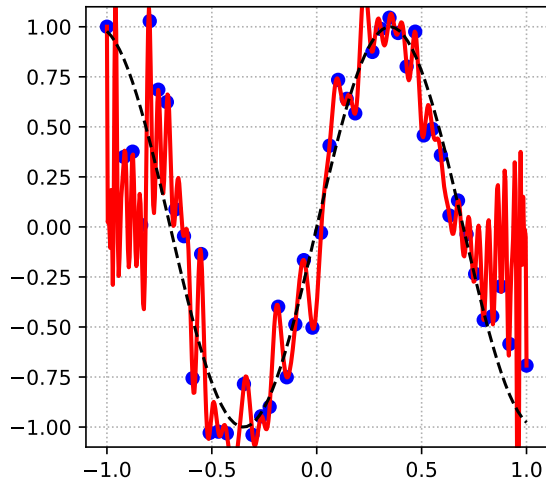


Figure 11: Кривые обучающего риска (пунктир) и тестового риска (сплошная линия). (a) Классическая U-образная кривая риска, возникающая из компромисса смещение–дисперсия. (b) Кривая двойного спуска, которая включает в себя U-образную кривую риска (т.е. «классический» режим) вместе с наблюдаемым поведением при использовании функциональных классов большой ёмкости (т.е. «современный» интерполирующий режим), разделённые порогом интерполяции. Предикторы правее порога интерполяции имеют нулевой обучающий риск.

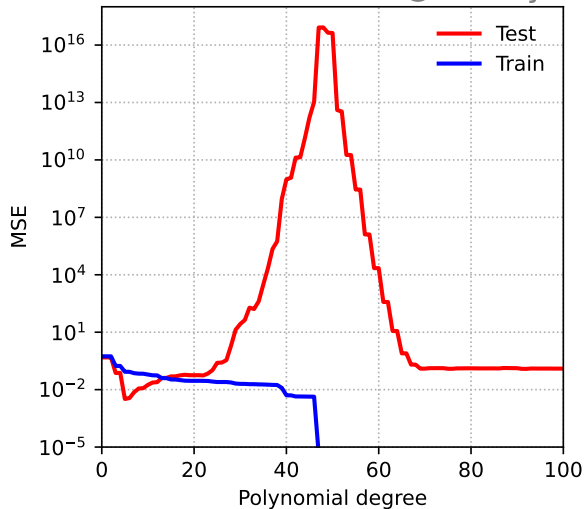
⁵Belkin, Mikhail, et al. "Reconciling modern machine-learning practice and the classical bias–variance trade-off." (2019)

Двойной спуск

Polynomial Fitting



@fminxyz



Shampoo⁶

Расшифровывается как **Stochastic Hessian-Approximation Matrix Preconditioning for Optimization Of deep networks**. Метод, вдохновлённый оптимизацией второго порядка и разработанный для крупномасштабного глубокого обучения.

Основная идея: Приближает полноматричный предобусловитель AdaGrad с помощью эффективных матричных структур, а именно произведений Кронекера.

Для матрицы весов $W \in \mathbb{R}^{m \times n}$ обновление включает предобусловливание с использованием приближений матриц статистики $L \approx \sum_k G_k G_k^T$ и $R \approx \sum_k G_k^T G_k$, где G_k — градиенты.

Упрощённая схема:

1. Вычислить градиент G_k .

Замечания:

Shampoo⁶

Расшифровывается как **Stochastic Hessian-Approximation Matrix Preconditioning for Optimization Of deep networks**. Метод, вдохновлённый оптимизацией второго порядка и разработанный для крупномасштабного глубокого обучения.

Основная идея: Приближает полноматричный предобусловитель AdaGrad с помощью эффективных матричных структур, а именно произведений Кронекера.

Для матрицы весов $W \in \mathbb{R}^{m \times n}$ обновление включает предобусловливание с использованием приближений матриц статистики $L \approx \sum_k G_k G_k^T$ и $R \approx \sum_k G_k^T G_k$, где G_k — градиенты.

Упрощённая схема:

1. Вычислить градиент G_k .
2. Обновить статистику $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$.

Замечания:

Shampoo⁶

Расшифровывается как **Stochastic Hessian-Approximation Matrix Preconditioning for Optimization Of deep networks**. Метод, вдохновлённый оптимизацией второго порядка и разработанный для крупномасштабного глубокого обучения.

Основная идея: Приближает полноматричный предобусловитель AdaGrad с помощью эффективных матричных структур, а именно произведений Кронекера.

Для матрицы весов $W \in \mathbb{R}^{m \times n}$ обновление включает предобусловливание с использованием приближений матриц статистики $L \approx \sum_k G_k G_k^T$ и $R \approx \sum_k G_k^T G_k$, где G_k — градиенты.

Упрощённая схема:

1. Вычислить градиент G_k .
2. Обновить статистику $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$.
3. Вычислить предобусловители $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$. (Обратный матричный корень)

Замечания:

Shampoo⁶

Расшифровывается как **Stochastic Hessian-Approximation Matrix Preconditioning for Optimization Of deep networks**. Метод, вдохновлённый оптимизацией второго порядка и разработанный для крупномасштабного глубокого обучения.

Основная идея: Приближает полноматричный предобусловитель AdaGrad с помощью эффективных матричных структур, а именно произведений Кронекера.

Для матрицы весов $W \in \mathbb{R}^{m \times n}$ обновление включает предобусловливание с использованием приближений матриц статистики $L \approx \sum_k G_k G_k^T$ и $R \approx \sum_k G_k^T G_k$, где G_k — градиенты.

Упрощённая схема:

1. Вычислить градиент G_k .
2. Обновить статистику $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$.
3. Вычислить предобусловители $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$. (Обратный матричный корень)
4. Обновление: $W_{k+1} = W_k - \alpha P_L G_k P_R$.

Замечания:

- Стремится учесть информацию о кривизне более эффективно, чем методы первого порядка.

Shampoo⁶

Расшифровывается как **Stochastic Hessian-Approximation Matrix Preconditioning for Optimization Of deep networks**. Метод, вдохновлённый оптимизацией второго порядка и разработанный для крупномасштабного глубокого обучения.

Основная идея: Приближает полноматричный предобусловитель AdaGrad с помощью эффективных матричных структур, а именно произведений Кронекера.

Для матрицы весов $W \in \mathbb{R}^{m \times n}$ обновление включает предобусловливание с использованием приближений матриц статистики $L \approx \sum_k G_k G_k^T$ и $R \approx \sum_k G_k^T G_k$, где G_k — градиенты.

Упрощённая схема:

1. Вычислить градиент G_k .
2. Обновить статистику $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$.
3. Вычислить предобусловители $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$. (Обратный матричный корень)
4. Обновление: $W_{k+1} = W_k - \alpha P_L G_k P_R$.

Замечания:

- Стремится учесть информацию о кривизне более эффективно, чем методы первого порядка.
- Вычислительно дороже Adam, но может сходиться быстрее или к лучшим решениям по числу шагов.

Shampoo⁶

Расшифровывается как **Stochastic Hessian-Approximation Matrix Preconditioning for Optimization Of deep networks**. Метод, вдохновлённый оптимизацией второго порядка и разработанный для крупномасштабного глубокого обучения.

Основная идея: Приближает полноматричный предобусловитель AdaGrad с помощью эффективных матричных структур, а именно произведений Кронекера.

Для матрицы весов $W \in \mathbb{R}^{m \times n}$ обновление включает предобусловливание с использованием приближений матриц статистики $L \approx \sum_k G_k G_k^T$ и $R \approx \sum_k G_k^T G_k$, где G_k — градиенты.

Упрощённая схема:

1. Вычислить градиент G_k .
2. Обновить статистику $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$.
3. Вычислить предобусловители $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$. (Обратный матричный корень)
4. Обновление: $W_{k+1} = W_k - \alpha P_L G_k P_R$.

Замечания:

- Стремится учесть информацию о кривизне более эффективно, чем методы первого порядка.
- Вычислительно дороже Adam, но может сходиться быстрее или к лучшим решениям по числу шагов.
- Требуется аккуратной реализации для достижения эффективности (например, эффективное вычисление обратных матричных корней, работа с большими матрицами).

Shampoo⁶

Расшифровывается как **Stochastic Hessian-Approximation Matrix Preconditioning for Optimization Of deep networks**. Метод, вдохновлённый оптимизацией второго порядка и разработанный для крупномасштабного глубокого обучения.

Основная идея: Приближает полноматричный предобусловитель AdaGrad с помощью эффективных матричных структур, а именно произведений Кронекера.

Для матрицы весов $W \in \mathbb{R}^{m \times n}$ обновление включает предобусловливание с использованием приближений матриц статистики $L \approx \sum_k G_k G_k^T$ и $R \approx \sum_k G_k^T G_k$, где G_k — градиенты.

Упрощённая схема:

1. Вычислить градиент G_k .
2. Обновить статистику $L_k = \beta L_{k-1} + (1 - \beta) G_k G_k^T$ и $R_k = \beta R_{k-1} + (1 - \beta) G_k^T G_k$.
3. Вычислить предобусловители $P_L = L_k^{-1/4}$ и $P_R = R_k^{-1/4}$. (Обратный матричный корень)
4. Обновление: $W_{k+1} = W_k - \alpha P_L G_k P_R$.

Замечания:

- Стремится учесть информацию о кривизне более эффективно, чем методы первого порядка.
- Вычислительно дороже Adam, но может сходиться быстрее или к лучшим решениям по числу шагов.
- Требуется аккуратная реализация для достижения эффективности (например, эффективное вычисление обратных матричных корней, работа с большими матрицами).
- Существуют варианты для тензоров различных форм (например, свёрточные слои).

$$\begin{aligned}
 W_{t+1} &= W_t - \eta(G_t G_t^\top)^{-1/4} G_t (G_t^\top G_t)^{-1/4} \\
 &= W_t - \eta(US^2U^\top)^{-1/4} (USV^\top)(VS^2V^\top)^{-1/4} \\
 &= W_t - \eta(US^{-1/2}U^\top)(USV^\top)(VS^{-1/2}V^\top) \\
 &= W_t - \eta US^{-1/2}SS^{-1/2}V^\top \\
 &= W_t - \eta UV^\top
 \end{aligned}$$

⁷K. Jordan blogpost "Muon: An optimizer for hidden layers in neural networks". 2024.

⁸J. Bernstein blogpost "Deriving Muon". 2025.

⁹Kovalev, D. (2025). Understanding Gradient Orthogonalization for Deep Learning via Non-Euclidean Trust-Region Optimization. arXiv preprint arXiv:2503.12645.

Сравнение Muon и AdamW на логистической регрессии

🔗 Простое сравнение Muon и AdamW на задаче логистической регрессии

Дополнительные материалы

-  Д. Ветров «Удивительные свойства ландшафта функции потерь в переопараметризованных моделях»

Дополнительные материалы

-  Д. Ветров «Удивительные свойства ландшафта функции потерь в переопараметризованных моделях»
-  В. Голошапов «О чём на самом деле не грокинг»

Дополнительные материалы

-  Д. Ветров «Удивительные свойства ландшафта функции потерь в переопараметризованных моделях»
-  В. Голошапов «О чём на самом деле не грокинг»
-  Д. Ковалёв «Understanding Gradient Orthogonalization for Deep Learning via Non-Euclidean Trust-Region Optimization» (Теория, лежащая в основе Muon)